START HERE / SAVE FOR LATER

Your AI camera, explained simply.

A complete, beginner-friendly architecture for a free live object-detection website.

THE WHOLE IDEA IN ONE LINE

Camera -> AI looks at a frame -> boxes appear on objects

Your phone or computer does the detecting. GitHub Pages only delivers the website files. No paid API is needed for the main feature.

What you will find inside

- **A big flowchart**
  See every part and where the arrows go.
- **Plain-English explanations**
  Know what each little piece actually does.
- **A future build checklist**
  Come back later and build it in small steps.

Version 1.0 | November 2026 | Local-first / GitHub Pages

01 / MAP

The big picture

Follow the arrows from top to bottom. This is the entire application in a nutshell.

1
OPEN THE WEBSITE
GitHub Pages sends your HTML, JavaScript, styles, and AI model files.

2
TAP START CAMERA
The browser asks: "Can this website use your camera?"

3
LIVE VIDEO APPEARS
You see the camera preview. The picture is still on your device.

4
PICK A FRAME
Every so often, choose one picture from the moving video.

5
AI CHECKS THAT FRAME
The local model guesses: "bottle", "chair", "person" ...

6
TRACK + DRAW
Give objects temporary IDs; draw names, scores, and boxes.

7
REPEAT WHILE RUNNING
Look at another new frame. Stop when the user taps Stop.

Optional side path: "Explain this scene"
Only when you ask, the app can send ONE snapshot to a protected cloud AI service. The main detection loop never needs it.

02 / PARTS

What each piece actually does

Think of the app as a tiny team. Each member has just one job.

01

GitHub Pages = delivery truck

It stores your website files and sends them to the visitor. It does NOT do the camera thinking.

02

Browser = the room

Chrome, Safari, or another browser runs the app on the visitor's device.

03

Camera = the eyes

It provides the live video, but only after the user gives permission.

04

Frame sampler = the pause button

A video has many pictures per second. This part chooses when the AI should inspect one.

05

AI model = the guesser

A trained file recognizes known object categories and gives a box plus a confidence score.

06

Tracker = the name tag giver

It tries to decide if the bottle seen now is the same bottle from the last frame.

07

Canvas overlay = the marker pen

It draws rectangles, names, scores, and IDs on top of the video.

08

Control panel = the remote

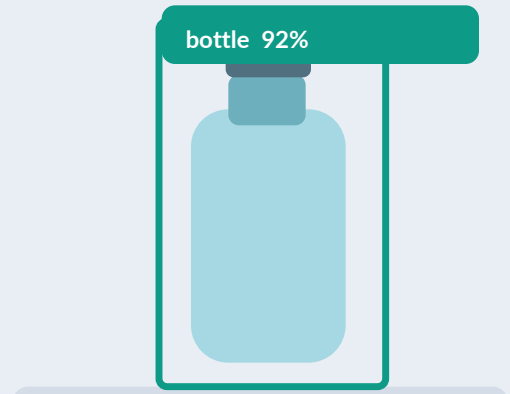
You choose camera, speed, confidence cutoff, and Start/Stop.

Easy rule: **The website draws and thinks locally; the hosting service just hands out files.**

03 / EXAMPLE

Follow one bottle through the app

Pretend you point the camera at a water bottle. Here is exactly what happens.



The box is a drawing over the video - it is not part of the camera picture.

1. **Camera sees it**
   A video stream contains lots of pictures.
2. **Sampler picks a picture**
   Maybe one frame every 100 milliseconds.
3. **Model looks**
   The model finds a likely bottle-shaped region.
4. **Model answers**
   Class: bottle. Score: 0.92. Box: x, y, width, height.
5. **Overlay draws it**
   A green rectangle and "bottle 92%" appear.
6. **Tracker remembers**
   The next frame can reuse the same temporary ID.

What the AI returns (in plain English)

Name of object	bottle
Confidence	0.92 = 92% model score; not a promise
Location	a rectangle (x, y, width, height)

Two important things

It can be wrong. A score is the model's confidence, not proof. Small, blurry, dark, or partly hidden objects can be missed.

It recognizes known categories. A general model might say "bottle," but not the exact brand or unique product name.

04 / CAMERA

How Start, Stop, and camera switching work

The browser guards access to your camera. Our code must politely ask first.

The camera life cycle

1. **Page opens**
   Show a clean screen with a Start Camera button. Do not start recording automatically.
2. **User taps Start**
   Ask the browser for camera access (navigator.mediaDevices.getUserMedia).
3. **User approves**
   Connect the camera stream to the live preview and load the detector.
4. **AI loop starts**
   Take new frames, detect objects, track them, and update the canvas overlay.
5. **User switches camera**
   Stop the old stream; request the other camera; update sizes and mirror rules.
6. **User taps Stop**
   Cancel the processing loop and stop every camera track. Clear the boxes.

What if something goes wrong?

Permission denied	Explain how to enable camera access in browser settings.
No camera exists	Show a friendly message instead of a blank black screen.
Camera already busy	Ask user to close the other app or try another camera.
Model download fails	Offer Retry; keep the camera preview useful.
Phone rotated	Resize/recalculate boxes so labels stay on the objects.

05 / FILES

Which files do we build?

React makes the screen. Small helper files do one job each. Here is a clean starter layout.

```
ai-camera/
  public/models/          AI model files live here
  src/components/
    CameraView.tsx        shows camera video
    DetectionOverlay.tsx  draws rectangles
    ControlPanel.tsx      Start, Stop, settings
  src/vision/
    camera.ts             opens and closes camera
    detector.ts           asks model to detect objects
    frameScheduler.ts     picks the next frame
    tracker.ts            keeps temporary object IDs
  src/App.tsx             connects everything
  src/main.tsx            starts the website
  vite.config.ts          GitHub Pages folder setting
  .github/workflows/      publishes the finished site
```

Start the project when you are ready

```
npm create vite@latest ai-camera -- --template react-ts

cd ai-camera && npm install

npm install @mediapipe/tasks-vision

npm run dev
```

Do not install everything at once. Begin with MediaPipe only. Add ONNX Runtime Web later, if you actually decide to use a second model.

06 / SPEED

How to keep the camera feeling smooth

The secret is simple: show video often, but ask the AI to work only as fast as the device can handle.

Camera: ~30 pictures per second

AI: maybe 5-20 checks per second depending on device

The preview can stay fluid even if detection updates less frequently.

Speed tricks, without complicated words

1. **Use a smaller picture for AI**
   The camera can display a clear video while the AI checks a 320 x 320 copy.
2. **Never make a line of waiting frames**
   If the AI is busy, skip the old picture. Use the latest picture next time.
3. **Load the model once**
   Loading the same model every frame wastes time and battery.
4. **Keep heavy work off the screen**
   Use a Web Worker where supported so tapping buttons stays responsive.
5. **Slow down on hot/slow phones**
   If detection takes too long, reduce AI checks per second or model size.
6. **Cache the files**
   After the first download, cached models can help with speed and offline use.

Phone details that are easy to forget

Portrait vs landscape: the video may be cropped; a box must be mapped to the part actually shown. **Front camera:** mirrored video needs mirrored boxes. **iPhone:** test Safari, not just desktop Chrome. **Battery:** add an Eco mode.

07 / CLOUD

What about API keys and Gemini?

Good news: the camera detector needs NO key. Cloud AI is a separate extra feature.

NORMAL MODE	EXTRA (IF ENABLED)
Live camera -> local AI Free; images stay on device	**One snapshot -> safe proxy -> AI** Only when user chooses to send

Never make your API key public

1. **Do not paste the secret into the website**
   Anyone can inspect public JavaScript files, including files hosted on GitHub Pages.
2. **Use a separate protected backend (later)**
   A serverless function can hold the key and check requests before contacting a cloud model.
3. **Keep the extra feature optional**
   No internet or no free API credits? The normal local detector still works.
4. **Show a clear privacy choice**
   Explain when a snapshot leaves the device. Do not quietly upload every frame.

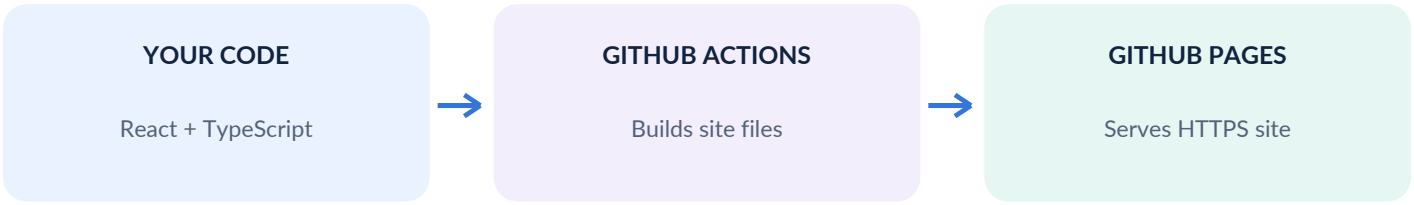
What does "free" mean here?

Core object detection: no usage-based API charges. **GitHub Pages:** free within GitHub's usage and repository rules. **Gemini/serverless:** some services have free allowances, but quotas and terms can change. Set spending limits and never rely on a permanent unlimited free tier.

08 / DEPLOY

How the website goes on GitHub Pages

Publishing means taking the built website and putting its ready-to-open files online.



Publishing checklist

1. **Create a repository**
   Name it ai-camera. Keep the code, models, and this architecture guide together.
2. **Push your project files**
   Commit the source files, package file, and downloaded open-license model weights.
3. **Set the Vite base path**
   For a project repo called ai-camera, use /ai-camera/ as the base path.
4. **Set up the GitHub Actions workflow**
   On a push to main: install dependencies, run the build, and publish dist/.
5. **Enable Pages in repo settings**
   Choose the GitHub Actions deployment source, not a made-up API server.
6. **Open on a real phone**
   Check camera permission, loading, Safari/Android behavior, and HTTPS.

Your future URL will look like:

https://YOUR-USERNAME.github.io/ai-camera/

09 / SAVE THIS

Your "build it later" cheat sheet

You do not need to build everything at once. Follow the small wins in order.

PHASE 1	**Camera works**	A Start/Stop button and live video on phone and PC.
PHASE 2	**AI sees objects**	Model loads; boxes, names, and confidence scores appear.
PHASE 3	**Make it reliable**	Camera switching, error messages, rotation, and stable tracking.
PHASE 4	**Make it faster**	Smaller frames, skip busy frames, Eco mode, Workers where possible.
PHASE 5	**Publish it**	Upload source to GitHub; build and host on Pages; test HTTPS.
PHASE 6	**Add extras later**	Optional YOLO, saved snapshots, Gemini with secure key proxy.

Tiny dictionary

Frame	One still picture inside video.	**Model**	The trained AI file that guesses objects.
Inference	The moment the AI looks at a frame.	**Confidence**	A score for how sure the model seems.
Bounding box	The rectangle showing where an object is.	**Latency**	The time AI needs to answer.
API key	Secret code for calling an online service.	**PWA**	A website that can behave more like an installed app.

Remember the main rule

Start with local MediaPipe detection. Make it work, then make it smooth, then make it fancy. Keep cloud AI optional.